

# 06 · Multicore-Echtzeit

AMP, STM32H755 und Shared Memory · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 55

## Willkommen in der Multicore-Königsklasse!

- Herzlich willkommen zum 6. Termin unserer Vorlesung
- Die letzten 5 Wochen: Single-Core-Echtzeit (Scheduling, IPC, Interrupts)
- Heute heben wir die Komplexität auf das nächste Level: Multicore-Systeme
- Zwei unterschiedliche Prozessorkerne in einem Chip
- Praxis am Nachmittag: zweites „Gehirn“ des STM32H755 aufwecken

## Notizen

Guten Morgen und herzlich willkommen in der Königsklasse der Embedded-Programmierung! In den letzten fünf Wochen: Task-Scheduling, Queues, Hardware-Interrupts über Semaphoren — alles auf einem Prozessorkern. Heute: Multicore-Architekturen, warum Ihr PC anders tickt als ein Steuergerät, und wie wir auf dem STM32H755 zwei Kerne parallel und sicher betreiben.

Folie

2 / 55

## Agenda für heute

1. Recap: Die Grenzen der Single-Core-Architektur
2. Symmetrisches vs. Asymmetrisches Multiprocessing (SMP vs. AMP)
3. Hardware Deep-Dive: Dual-Core-Architektur des STM32H755
4. Boot-Prozess und Toolchain-Setup für zwei Kerne
5. Die Speicherarchitektur (RAM-Domänen)
6. *[Nach der Pause]* Shared Memory und Cache-Kohärenz
7. Ausblick Labor: Ping-Pong-Datenaustausch

## Notizen

Motivation für mehrere Kerne, dann SMP vs. AMP. Anschließend den STM32H755 zerlegen: M7 und M4, Boot und Speicheraufteilung. Nach der Pause: Cache-Kohärenz bei geteiltem Speicher. Am Nachmittag kommunizieren beide Kerne live.

## Recap: Die Grenzen des Single-Core

- Erinnern wir uns an die Schedulability-Tests (Termin 2)
- EDF-Schranke:  $\sum \frac{C_i}{T_i} \leq 1$  (100% Auslastung)
- Bei 100% CPU-Nutzung ist physikalisch Schluss
- Das Problem der Industrie:
  - Steuergeräte sollen immer mehr können (KI, Ethernet, GUI)
  - Gleichzeitig harte Timing-Regelkreise (100 s Raster)
- Wie bekommen wir mehr Rechenleistung in das System?

## Notizen

Woche 2: EDF kann die CPU zu 100% auslasten — mathematisches Limit. Ein modernes Automotive-Steuergerät soll Kamerabilder mit KI auswerten, Touch-Display bespielen und den Elektromotor mit Mikrosekunden-Präzision ansteuern. Ein einzelner Kern ist überfordert. Wir brauchen mehr Rechenleistung.

## Erhöhung der Taktfrequenz?

- Warum takten wir den Cortex-M4 nicht mit 5 GHz statt 240 MHz?
- Die Power Wall (Leistungsmauer):
  - Höherer Takt → höhere Spannung → exponentiell mehr Abwärme
  - Embedded Systems oft ohne Lüfter (z. B. vergossenes Motorsteuergerät)
- Die Memory Wall (Speichermauer):
  - RAM-Zugriffszeiten skalieren nicht mit CPU-Takten
  - Die CPU wartet auf Daten (Pipeline Stalls)

## Notizen

Intuitiv: Takt erhöhen. Physik: Power Wall — 5 GHz erzeugen massive Abwärme. Im PC: Kühler und Lüfter. Im Auto: oft am Getriebemotorblock, wasserdicht vergossen, 100 Grad Umgebungstemperatur ohne Lüfter. Memory Wall: selbst bei 5 GHz intern liefert der Arbeitsspeicher zu langsam — die CPU verbrennt Zeit mit Warten.

## Der Ausweg: Multicore-Architekturen

- Lösung: lieber mehrere langsame, effiziente Kerne als ein extrem heißer, schneller Kern
- Beispiele aus der Industrie:
  - Automotive Domain Controller: viele kleine Steuergeräte in einem Multicore-Chip
  - Industrie 4.0 (Edge Computing): ein Core steuert die Maschine (harte Echtzeit), ein anderer funkt in die Cloud (weiche Echtzeit)
- Multicore in der Echtzeit ist fundamental anders als in Ihrem Laptop

### Notizen

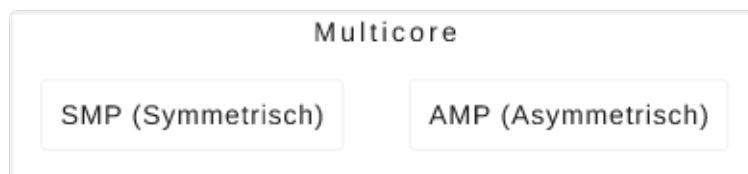
Die Lösung von Intel/AMD für den PC ist in der MCU-Welt angekommen: Multicore. Automotive: Domain Controller statt vieler Einzelprozessoren. Industrie 4.0: harte Echtzeit auf einem Kern, MQTT/OPC-UA auf dem anderen. Aber: Echtzeit-Multicore tickt völlig anders als der i7 in Ihrem Laptop.

## Multiprocessing-Architekturen

Zwei grundlegende Konzepte:

1. SMP (Symmetric Multiprocessing)
2. AMP (Asymmetric Multiprocessing)

Beide haben unterschiedliche Einsatzgebiete.



### Notizen

Es gibt nicht „die eine“ Multicore-Architektur. Zwei Familien: SMP und AMP. Beide lösen Rechenlast, aber auf unterschiedlichen Wegen. Für Ihre Karriere: Unterschied kennen und wissen, warum AMP in der Echtzeit oft bevorzugt wird.

## Symmetrisches Multiprocessing (SMP)

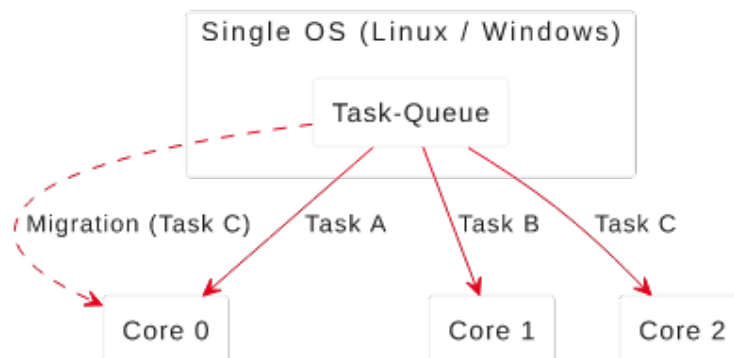
- Die PC- und Smartphone-Welt (Windows, Linux, Android, iOS)
- Architekturmerkmale:
  - Alle Kerne physisch identisch (z. B. 4× Cortex-A53)
  - Gemeinsamer physikalischer Speicher (RAM)
  - Genau ein Betriebssystem über alle Kerne
  - Das OS orchestriert alle Kerne zentral

### Notizen

SMP kennen Sie aus Alltag: Handy und Laptop. „Symmetrisch“: identische Kerne, gleicher Befehlssatz, gleicher Takt, gemeinsamer RAM. Darüber liegt ein einziges OS (Windows, Linux-Kernel). Das OS sieht alle Kerne und verteilt die Arbeit.

## Wie funktioniert ein SMP-Scheduler?

- Das OS hat eine Ready-Warteschlange für Tasks
- Load-Balancing: Tasks dynamisch auf den Kern mit wenig Last
- Task-Migration: Millisekunde 1 auf Core 0, Millisekunde 2 auf Core 1



### Notizen

Der SMP-Scheduler betreibt Load-Balancing. Word, Excel, Browser: OS weist Threads freien Kernen zu. Task-Migration: Thread startet auf Core 0, wird pausiert, läuft später auf Core 1 weiter — Register wandern mit. Fantastische Gesamtauslastung.

## Warum SMP für harte Echtzeit oft scheitert

- SMP ist genial für Durchsatz, aber ein Albtraum für Determinismus
- Latenz-Probleme:
  - Task-Migration kostet Zeit (Register von Kern zu Kern)
  - Cache-Thrashing: L1-Cache des Zielkerns ist leer → Cache-Misses
  - Contention: mehrere Kerne auf demselben RAM-Bus → Mikro-Staus
- Ergebnis: WCET explodiert und wird unberechenbar

### Notizen

Warum keine Airbag-Steuerung mit SMP? Tod für Determinismus. Migration: Register kopieren und L1-Cache leer → Jitter. Bus-Staus bei parallelem RAM-Zugriff. Die WCET für den Schedulability-Test wird unberechenbar.

## Asymmetrisches Multiprocessing (AMP)

- Die Embedded-Welt (Steuergeräte, Motor-Regler)
- Architekturmerkmale:
  - Kerne oft unterschiedlich (z. B. Cortex-M7 und Cortex-M4)
  - System strikt partitioniert
  - Jeder Kern führt eigenes Programm / OS aus
  - Kerne wissen beim Start oft nichts voneinander

### Notizen

Daher AMP in der Echtzeit. Asymmetrisch: oft ungleiche Kerntypen — wuchtiger Kern für Rechnen, präziser Kern für Steuerung. Kein Über-Betriebssystem: zwei unabhängige Programme. Core A und Core B booten getrennt.

## Software-Aufteilung in AMP

Core 1 (z. B. Cortex-M7)

- OS: Embedded Linux oder Bare-Metal
- Job: GUI, Netzwerk (TCP/IP), ML
- Fokus: Durchsatz (weiche Echtzeit)

Core 2 (z. B. Cortex-M4)

- OS: FreeRTOS oder AUTOSAR
- Job: PWM, Encoder, PID-Regler
- Fokus: Determinismus (harte Echtzeit)
- 100% CPU-Load auf Core 1 hat keine Auswirkung auf das Timing von Core 2

### Notizen

Bestes aus beiden Welten: rechenstarker Kern mit Linux/Netzwerk (Timing darf schwanken), kleiner Kern mit FreeRTOS und Motor-PWM. Isolation fast perfekt: hängt Linux bei 100% Auslastung, tuckert der FreeRTOS-Regler jitterfrei weiter — Safety per Hardware-Design.

## Wie kommunizieren AMP-Kerne?

- Kein gemeinsames OS → keine normale FreeRTOS-Queue von Core 1 nach Core 2
- Kommunikation über Hardware-Mechanismen:
  - Message Buffers (Hardware-Mailboxes): Schreiben löst Interrupt am anderen Kern aus
  - Hardware-FIFOs: echte Puffer zwischen den Kernen
  - Shared Memory (SRAM): vereinbarter RAM-Bereich mit physischem Zugriff beider Kerne
- Heute: Shared Memory — flexibel, aber gefährlich

### Notizen

Wie reden die Kerne? `osMessageQueuePut` geht nicht — FreeRTOS auf Core 2 kennt die Datenstrukturen von Bare-Metal auf Core 1 nicht. Hardware hilft: Queues oder Shared Memory. Beide Kerne greifen auf denselben RAM-Block zu. Schnell, aber tückisch bei der Datenintegrität.

## Zusammenfassung Block 1

| Eigenschaft    | SMP (Symmetrisch)                  | AMP (Asymmetrisch)              |
|----------------|------------------------------------|---------------------------------|
| Kerne          | Identisch                          | Oft heterogen                   |
| Betriebssystem | Ein gemeinsames OS                 | Getrennte Programme / OS        |
| Lastverteilung | Dynamisch durch Scheduler          | Statisch durch Programmierer    |
| Determinismus  | Schlecht (Migration, Cache-Misses) | Hervorragend (strikte Trennung) |
| Beispiel       | Laptop, Raspberry Pi               | STM32H755, NXP i.MX RT          |

### Notizen

SMP: identische Kerne, ein OS, Meister des Durchsatzes. AMP: oft ungleiche Kerne, strikte Isolation, Meister des Determinismus. Wir bauen fast ausschließlich AMP-Systeme. Als Nächstes: der STM32H755 im Detail.

## Hardware Deep-Dive: Der STM32H755

- Ein Silizium-Plättchen (Die), zwei getrennte Prozessorkerne



- Trotzdem teilen sie sich Pins, Taktquellen und Speicher
- Wie organisiert STMicroelectronics dieses Chaos?

### Notizen

Der STM32H755 ist ein Flaggschiff der AMP-Architektur. Ein Gehäuse, zwei Herzen: Cortex-M7 und Cortex-M4. Logisch getrennt, physisch im selben Package: Strom, Clocks, Pins teilen. Core 1 darf Pin A5 nicht High setzen, während Core 2 ihn Low zieht.

## Der Cortex-M7 (Das Muskelpaket)

- Unser Core 1 (Master)
- Specs: bis 480 MHz, 6-stufige superskalare Pipeline
- Superskalar: zwei Instruktionen in einem Taktzyklus möglich
- Caches:
  - L1 Data Cache (16 KB)
  - L1 Instruction Cache (16 KB)
- FPU: Double-Precision (64-Bit)
- Ziel: Rechenleistung, schnelle Netzwerk-Verarbeitung

### Notizen

Core 1, der Cortex-M7: Muskelpaket bis 480 MHz, superskalar mit parallelen Rechenwerken. L1-Caches für Daten und Code, weil RAM für 480 MHz zu langsam ist. 64-Bit-FPU. Gebaut für Filter, Ethernet und Touch-Displays.

## Der Cortex-M4 (Das Präzisionswerkzeug)

- Unser Core 2
- Specs: bis 240 MHz, 3-stufige Pipeline
- Wichtigster Unterschied: keine Caches!
- Ausführungszeit eines Assembler-Befehls ist absolut deterministisch
- FPU: Single-Precision (32-Bit)
- Ziel: harte Echtzeit-Regelung, enge Anbindung an Timer und ADC
- Hier lassen wir FreeRTOS laufen

### Notizen

Der Cortex-M4: halb so schnell, simplere Pipeline. Entscheidend: keine Caches. Jeder Speicherzugriff dauert immer gleich viele Takte — Timing auf die Nanosekunde vorherberechenbar. Skalpell neben dem Vorschlaghammer. Ideal für Motoren und Leistungselektronik. FreeRTOS läuft heute Nachmittag exklusiv auf dem M4.

## Die Bus-Matrix (AHB / AXI)

- Kerne, Speicher und Peripherie: keine einzelne Leitung, sondern eine Matrix
- AXI-Bus-Matrix: Netzwerk aus Schaltern
- Vorteil: M7 liest Flash, während M4 eine GPIO-LED schaltet — ohne Blockade
- Blockaden nur, wenn beide Cores im selben Takt dasselbe Ziel treffen

### Notizen

Früher: ein Bus, eine Hauptstraße. Der STM32H7 nutzt eine AXI-Bus-Matrix — Autobahnkreuze und Brücken. M7 auf Flash und M4 auf GPIO kreuzen sich nicht. Stau nur bei gleichzeitigem Zugriff auf dasselbe Modul; die Matrix regelt Vorfahrt.

## Die 3 Power-Domänen (D1, D2, D3)

- D1 (High-Performance Domain):
  - Cortex-M7, AXI-SRAM, High-Speed USB, Ethernet
- D2 (Real-Time Domain):
  - Cortex-M4, SRAM 1/2/3, Timer, ADCs, UARTs
- D3 (Low-Power Domain):
  - RTC, SRAM 4
  - Läuft oft weiter, wenn D1 und D2 schlafen

### Notizen

Drei Kontinente: D1 (Partyzone mit M7 und Gigabit), D2 (Echtzeit-Zone mit M4 und Timern), D3 (Low-Power). Im Batteriebetrieb können D1 und D2 abgeschaltet werden — D3 hält Uhr und SRAM4 mit Mikroampere am Leben.

## Taktversorgung (RCC)

- Beide Kerne haben eigene Phasenregelkreise (PLLs)
- M7 kann mit 400 MHz laufen, M4 mit 200 MHz
- Achtung im CubeMX Configurator:
  - Änderungen am System Clock beeinflussen meist beide Kerne
  - Stoppt Core 1 die PLL, crasht Core 2

### Notizen

Ähnliche Teilung bei RCC: eigene Teiler und PLLs. M7 auf 480 MHz, M4 auf 100 MHz ist möglich. Falle: der externe Quarz ist gemeinsam. Schaltet M7 den Hauptoszillator ab, bleibt dem M4 das Herz stehen — einer muss Boss bleiben.

## Der Boot-Prozess: Wer weckt wen?

- Beim Einschalten starten nicht beide Kerne
- Cortex-M7 ist der Master und bootet zuerst
- Cortex-M4 ist der Slave und bleibt im Reset (Hold)
- Aufgabe des M7:
  - System-Clock hochfahren
  - RAM und Peripherie stabilisieren
  - Hardware-Schalter umlegen, der den M4 entfesselt

### Notizen

Häufige Anfängerfrage: Wer startet zuerst? Nur der M7. Der M4 bleibt im Reset. M7 konfiguriert Taktbaum und Bus-Matrix, legt dann den Schalter um — M4 springt in seine eigene `main()`.

## Hardware-Register: Den M4 wecken

- Schalter im Power-Management-Register (PWR)
- In der `main.c` des M7:

```
// Weckt den Cortex-M4 auf  
HAL_PWREx_ReleaseCore(PWR_CORE_CPU2);
```

- Ohne diese Zeile läuft FreeRTOS auf dem M4 nie

### Notizen

Diesen Schalter betätigen Sie heute Nachmittag selbst. In der `M7-main.c`: `HAL_PWREx_ReleaseCore(PWR_CORE_CPU2)`. Blinken die M4-LEDs nicht: zuerst diese Zeile prüfen.

## Pin-Zuweisung im Dual-Core

- Dem Chip ist egal, wer einen GPIO-Pin schaltet
- Gefahr: M7 und M4 konfigurieren denselben Pin → Register-Konflikte
- Im CubeMX Configurator:
  - Pin Context Assignment → ARM Cortex-M7 oder ARM Cortex-M4
  - Codegenerator schreibt `HAL_GPIO_Init` nur in die `main.c` des zugewiesenen Cores

### Notizen

Beide können auf GPIO schreiben. M7 als Ausgang und M4 als Eingang am selben Pin kracht. In CubeIDE: Pin Context Assignment. Der Generator schreibt die Init nur in den Ordner des zugewiesenen Kerns — saubere Architektur.

## Toolchain und Projektstruktur

- Dual-Core-Projekt in CubeIDE: drei Ordner
  - `Project_CM7`: eigene `main.c`, eigene Linker-Skripte
  - `Project_CM4`: isolierte Code-Basis für den M4
  - `Project_Common`: gemeinsame Dateien (z. B. HAL-Treiber)
- Beim Kompilieren entstehen zwei unabhängige `.elf` / `.bin`-Dateien

### Notizen

Drei Ordner statt einem: CM7 und CM4 mit eigener `main.c`, Interrupt-Tabellen und Linker-Skripten — zwei C-Projekte. Common nur für gemeinsame Treiber. GCC wirft zwei Firmware-Dateien aus: eine für Master, eine für Slave.

## Debugging im AMP-System

- ST-Link unterstützt Multicore-Debugging
- Debug startet zwei GDB-Instanzen parallel
- Zwei Konsolen: Breakpoints von M7 und M4 umschalten
- Flashen: Programm 1 in Bank 1, Programm 2 in Bank 2

### Notizen

Der ST-Link fragt beide Kerne per JTAG ab. Debug flasht beide Programme in getrennte Flash-Banken und startet zwei Debug-Server. Breakpoint im FreeRTOS des M4 setzen, dann in den M7-Tab wechseln und zuschauen, wie Bare-Metal weiterläuft.

## Zusammenfassung Block 2

- STM32H755: High-Performance-Kern (M7 mit Cache) und Echtzeit-Kern (M4 ohne Cache)
- Bus-Matrix routet Zugriffe kreuzungsfrei — außer bei exakt demselben Ziel
- M7 bootet zuerst und weckt den M4
- Zwei unabhängige Firmware-Dateien aus zwei Code-Ordern

## Notizen

Zusammenfassung Hardware: schnelles und determiniertes Herz, Bus-Matrix gegen Staus, M7 als Boss, Software wie zwei separate Projekte. Fragen zur Hardware, bevor wir zum Speicher und zur Kommunikation gehen?

## Folie

26 / 55

### Die Speicherarchitektur (RAM)

- Datenaustausch M7 → M4 braucht Shared Memory
- Fast 1 MB RAM, aber aufgeteilt:
  - AXI-SRAM (D1): klebt am M7 — schnell für M7, langsam für M4
  - SRAM 1/2/3 (D2): klebt am M4 — schnell für M4, langsam für M7
  - SRAM 4 (D3): unabhängig, oft für Peripherie oder Standby

## Notizen

AMP-Kerne reden über Shared Memory — aber es gibt nicht „den einen“ RAM. AXI-SRAM klebt am M7 (1 Takt für M7, teuer für M4 über die Bus-Matrix). SRAM 1–3 kleben beim M4 in D2.

## Folie

27 / 55

### Wo legen wir den Shared Memory an?

- Neutraler Ort, gut erreichbar für beide Kerne
- Best Practice: SRAM4 in der D3-Domäne
- Basis-Adresse: `0x38000000` (Größe: 64 KB)
- Ziel: C-Struct für Sensorwerte exakt an diese Adresse legen
- Wie zwingen wir den Compiler dazu?

## Notizen

Treffpunkt oft SRAM4 in D3: 64 KB ab `0x38000000`. Ein C-Struct darf nicht irgendwo im AXI-SRAM landen — wir nageln es bytengenau auf diese Adresse.

## Das Linker-Script (.ld)

- GCC erzeugt Maschinencode; der Linker legt physikalische Adressen fest
- Linker-Skripte beider Kerne anpassen:

```
/* Definition der Speicherbereiche */
MEMORY
{
    FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 1024K
    RAM_D1 (xrw)     : ORIGIN = 0x24000000, LENGTH = 512K
    /* NEU: Shared Memory */
    SHARED_RAM (xrw) : ORIGIN = 0x38000000, LENGTH = 64K
}
```

### Notizen

Das Linker-Script ist das Fundament. In beiden .ld-Dateien einen MEMORY-Bereich SHARED\_RAM mit Adresse 0x38000000 und 64 KB deklarieren — beide Kerne kennen den Bereich.

## Eine eigene Sektion anlegen

- Sektion (Container) im Shared-RAM anlegen:

```
SECTIONS
{
    /* Sektion .shared_memory */
    .shared_memory (NOLOAD) :
    {
        . = ALIGN(4);
        *(.shared_memory) /* Variablen mit diesem Tag */
    } > SHARED_RAM
}
```

- Extrem wichtig: (NOLOAD)
  - Ohne NOLOAD füllt der Startup-Code den RAM mit Nullen
  - M7 schreibt, M4 bootet 1 ms später → Shared Memory wird genullt

### Notizen

Sektion .shared\_memory anlegen. Entscheidend: NOLOAD. Ohne NOLOAD initialisiert der

Startup alle globalen Variablen mit 0. M7 schreibt Sensordaten, M4 bootet eine Millisekunde später und überschreibt den Shared Memory mit Nullen. NOLOAD verbietet dieses Nullen.

## Folie

30 / 55

### Zuweisung im C-Code (GCC-Attribute)

- Wir haben den RAM (.shared\_memory) im Linker-Script reserviert
- Wie sagen wir dem Compiler, dass unser Struct genau dort landen soll?
- Die Lösung: ein Compiler-spezifisches Attribut (\_\_attribute\_\_)
- Wir „nageln“ die Variable physisch auf die Adresse 0x38000000 fest

```
// Die Variable wird zwingend in  
// die .shared_memory Sektion gelegt!  
  
__attribute__((section(".shared_memory")))  
SharedData_t shared_data;
```

## Notizen

Wir haben den Grundstein im Linker-Script gelegt. Aber wenn wir in C einfach eine globale Variable deklarieren, packt der Compiler sie standardmäßig in die .bss oder .data-Sektion — irgendwo im schnellen AXI-SRAM. Um das zu verhindern, nutzen wir eine GCC-spezifische Anweisung: \_\_attribute\_\_((section("name"))). Wir hängen dieses Attribut direkt vor unsere Variablendeklaration. Wenn der Compiler diesen Code sieht, gibt er dem Linker die Anweisung: Diese Variable muss exakt in den Topf namens .shared\_memory geworfen werden. Und da dieser Topf fest auf die Adresse 0x38000000 gelinkt ist, wissen wir zu 100 %, dass beide Kerne auf dieselben Bytes zugreifen.

## Folie

31 / 55

### Das Schlüsselwort volatile

- Ein weiteres Problem beim Speichern: der C-Compiler-Optimizer (-O2 / -O3)
- Der Compiler analysiert den Code einer Task (z. B. auf dem M4)
- Er sieht eine while(shared\_data.m7\_ready == 0); -Schleife
- Er denkt: „In meiner Task ändert niemand diese Variable. Also lese ich sie nur 1× aus dem RAM und nutze danach für immer ein CPU-Register!“ → Endlosschleife!
- Lösung: volatile
- Sagt dem Compiler: Diese Variable kann sich jederzeit durch externe Hardware (oder einen anderen Kern) ändern! Lese sie bei JEDEM Zugriff zwingend frisch aus dem RAM!

## Notizen

Das nächste Problem ist hausgemacht durch clevere Software. Moderne C-Compiler sind darauf trainiert, Ihren Code rasend schnell zu machen. Wenn Sie auf dem M4 eine While-Schleife schreiben, die darauf wartet, dass der M7 ein Flag setzt, liest der Optimizer den Code. Er stellt fest: In dieser Task wird die Variable nie beschrieben — es sei sinnlos, sie in jedem Schleifendurchlauf aus dem RAM zu holen. Er liest sie einmal in ein CPU-Register und prüft ab sofort nur noch dieses Register. Da der M7 aber den RAM ändert und nicht das Register des M4, friert der M4 für immer ein. Um das zu verhindern, müssen wir die Variable als `volatile` deklarieren. Das zwingt den Compiler, den Wert jedes Mal frisch über die Bus-Matrix aus dem RAM abzuholen.

## Folie

32 / 55

### Reicht volatile für Multicore?

- Wir haben die Variable im richtigen SRAM platziert (`__attribute__`)
- Wir haben den Compiler gezwungen, immer aus dem RAM zu lesen (`volatile`)
- Sind wir jetzt sicher?
- NEIN! Absolut nicht
- `volatile` kontrolliert nur den Software-Compiler. Es kontrolliert nicht die Hardware-Caches der CPU!
- Hier betreten wir das gefährlichste Feld der Embedded-Entwicklung

## Notizen

Viele C-Programmierer denken: Prima, ich habe meine globale Variable als `volatile` deklariert, jetzt bin ich sicher vor Multicore-Problemen. Falsch! Absolut falsch. `volatile` ist ein reines Software-Konstrukt. Es sagt dem GCC-Compiler, dass er RAM-Zugriffe nicht wegoptimieren darf. Aber es sagt der physikalischen CPU-Hardware, genauer gesagt dem L1-Cache des Cortex-M7, absolut gar nichts. Die CPU darf und wird die Zugriffe hardwareseitig puffern. Und genau dieses Hardware-Puffern zerschießt uns die Multicore-Kommunikation.

## Folie

33 / 55

### Struktur der Ping-Pong Daten

- Für unser Labor bauen wir uns ein C-Struct, das beide Kerne kennen
- Beide Kerne binden dieselbe Header-Datei ein!

```
typedef struct {
    uint8_t  m7_ready;      // 1 = M7 hat Daten geschrieben
    uint8_t  m4_ready;      // 1 = M4 hat Daten geschrieben
    uint32_t payload_value; // Der eigentliche Wert (Payload)
} SharedData_t;

// Im Code (in CM7 und CM4):
__attribute__((section(".shared_memory")))
volatile SharedData_t shared_data;
```

### Notizen

Bevor wir uns dem Cache-Problem widmen, hier noch kurz die Datenstruktur für unser heutiges Labor. Wir bauen ein Struct namens `SharedData_t`. Es enthält ein Flag, mit dem der M7 signalisiert „Ich habe fertig geschrieben“, und ein Flag für den M4. Darunter liegt der eigentliche Payload-Wert. Dieses Struct deklarieren wir als `volatile` und legen es in die Shared-Memory-Sektion. Achten Sie darauf, dass diese Definition in einer Header-Datei liegt, die in beide Unterprojekte (CM4 und CM7) eingebunden wird. Sonst wissen die Kerne nicht, wie die Bytes im RAM angeordnet sind.

### Folie

34 / 55

### Zusammenfassung: Shared Memory

- AMP erfordert physikalischen Datenaustausch, da es kein gemeinsames Betriebssystem gibt
- Wir nutzen den SRAM4 (D3-Domäne), da er für beide Kerne gut erreichbar ist
- Die Adress-Zuweisung erfolgt über das Linker-Script (`.ld`) und `__attribute__((section))`
- Das Flag `NOLOAD` schützt den RAM vor dem Überschreiben beim Booten
- Das Keyword `volatile` stoppt gefährliche Compiler-Optimierungen

### Notizen

Fassen wir den ersten Teil zusammen. Wir haben die logische Struktur für unsere Inter-Core-Kommunikation gelegt. Wir nutzen den SRAM4 als toten Briefkasten. Wir weisen dem Linker den Weg und hindern den C-Startup-Code am Nullen der Daten. Wenn Sie das auf einem Cortex-M0 oder Cortex-M4 Dual-Core machen würden, wären Sie jetzt fertig. Aber wir haben einen Cortex-M7. Und der M7 bringt ein Feature mit, das uns jetzt das Leben zur Hölle macht.

## Was ist ein CPU-Cache?

- Die CPU (z. B. 400 MHz) ist extrem schnell
- Der Hauptspeicher (SRAM) ist langsam (z. B. braucht ein Zugriff 5 Taktzyklen)
- Der L1-Cache:
  - Ein winziger, aber unfassbar schneller SRAM direkt im Prozessorkern (1 Taktzyklus)
  - Wenn die CPU eine Variable aus dem RAM liest, kopiert die Hardware einen ganzen Block (Cache-Line, 32 Byte) in den Cache
  - Beim nächsten Lesen wird die Variable aus dem Cache geholt (blitzschnell!)

## Notizen

Was genau ist ein Cache? Der Cortex-M7 rechnet mit 400 Megahertz. Der Arbeitsspeicher antwortet aber vielleicht erst nach 5 Taktzyklen. Die CPU würde ständig warten. Deshalb hat der M7 direkt auf dem Silizium des Kerns einen kleinen Speicher, den Level-1-Cache. Wenn Ihr C-Code eine Variable liest, holt die Hardware nicht nur diese eine Variable aus dem lahmen RAM, sondern kopiert die umgebenden 32 Bytes in den pfeilschnellen Cache. Wenn Sie in der nächsten Code-Zeile nochmal auf die Variable zugreifen, fragt die CPU gar nicht mehr den RAM. Sie liest die Daten in einem einzigen Taktzyklus direkt aus dem Cache.

## Write-Through vs. Write-Back

- Was passiert, wenn der C-Code eine Variable schreibt (`shared_data.m7_ready = 1;`)?
- **Write-Through:**
  - Die CPU schreibt den Wert in den Cache UND sofort parallel durch in den Haupt-RAM
  - (Sicher, aber langsamer)
- **Write-Back** (Standard beim Cortex-M7):
  - Die CPU schreibt den Wert NUR in den internen Cache!
  - Der echte SRAM bleibt veraltet (Stale Data)
  - Der Cache schreibt die Daten erst in den RAM zurück (Eviction), wenn er Platz für neue Variablen braucht

## Notizen

Das Lesen ist unproblematisch. Aber was passiert beim Schreiben? Hier gibt es in der IT zwei Architekturen. Bei Write-Through schreibt der Cache jede Änderung sofort durch auf den großen SRAM. Das ist sicher, bremst den Bus aber aus. Moderne Prozessoren wie der Cortex-M7 nutzen deshalb standardmäßig Write-Back. Wenn Ihr Code `m7_ready = 1` ausführt, speichert die Hardware diese „1“ ausschließlich im winzigen L1-Cache des M7. Im echten SRAM auf der Platine steht weiterhin eine „0“. Der L1-Cache

behält das für sich, bis er irgendwann voll ist und Platz machen muss. Erst dann schreibt er die Daten in den echten RAM zurück. Und hier beginnt die Katastrophe für unsere Multicore-Kommunikation.

## Folie

37 / 55

### Das Drama der Cache-Kohärenz

- Das Szenario:
  - Der Cortex-M7 rechnet einen Wert aus und setzt das Flag `m7_ready = 1`
  - Dank Write-Back landet die 1 nur im L1-Cache des M7. Im SRAM4 steht weiterhin 0
  - Der Cortex-M4 (ohne Cache) fragt über die Bus-Matrix den SRAM4 ab
  - Der SRAM4 antwortet mit 0
- Ergebnis: Der M4 wartet ewig. Die Cores sind asynchron. Die Daten sind korumpiert!
- Dieses Problem nennt man einen *Cache Coherency Failure*

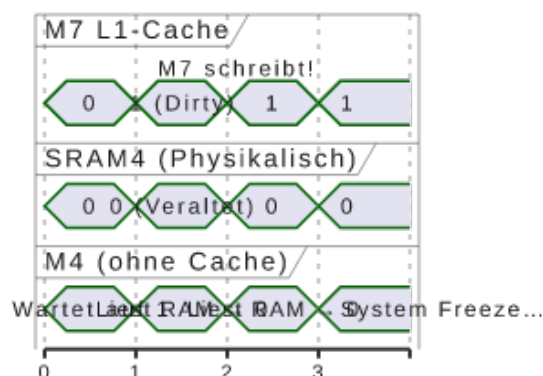
## Notizen

Lassen Sie uns das Drama Schritt für Schritt durchspielen. Der Cortex-M7 rechnet brav den Sensorwert aus und setzt das Bereit-Flag auf 1. Das Programm läuft fehlerfrei. Aber die Hardware puffert diesen Schreibvorgang im Write-Back-Cache. Im physikalischen SRAM4 steht immer noch eine Null. Eine Millisekunde später prüft der Cortex-M4 das Flag. Da der M4 keinen Cache hat, geht sein Lesebefehl über die Bus-Matrix direkt an den physikalischen SRAM4. Und was liefert der SRAM4 zurück? Die alte, veraltete Null! Der M4 denkt, der M7 hat noch gar nichts gerechnet und blockiert. Die Daten im System sind asynchron. Der Cache des M7 ist nicht kohärent mit dem Hauptspeicher.

## Folie

38 / 55

### Visualisierung: Cache Coherency Failure



- Der M4 sieht die neuen Daten des M7 nicht, da sie im L1-Cache „feststecken“

## Notizen

In diesem Timing-Diagramm sehen Sie exakt das, was Sie in einem Labor ohne Gegenmaßnahmen erleben würden. Bei  $t = 1$  schreibt der Cortex-M7-Code eine „1“. Der Wert ändert sich aber nur im obersten Graphen, im M7-Cache. Der Graph in der Mitte, der physikalische RAM, bleibt auf „0“. Der untere Graph, unser Cortex-M4, liest stoisch den RAM und liest immer wieder „0“. Das System friert effektiv ein, ohne dass der C-Compiler oder das RTOS irgendeinen Fehler gemacht hätten. Es ist ein reines Hardware-Problem.

## Folie

39 / 55

### HW-Lösung in PCs: Cache-Snooping

- Wie löst Ihr Intel-i7 im Laptop das Problem?
- Er nutzt Hardware-Snooping (z. B. MESI-Protokoll)
- Die Kerne „schnüffeln“ (Snoop) gegenseitig auf dem Bus
- Wenn Core 2 den RAM liest, bemerkt Core 1 das, unterbricht den Vorgang, schreibt schnell seinen Cache in den RAM und lässt Core 2 dann erst lesen
- Warum hat der STM32H755 das nicht?
- Snooping kostet extrem viel Siliziumfläche, viel Strom und (das Schlimmste) verursacht unvorhersehbare Bus-Latenzen → Gift für harte Echtzeit!

## Notizen

Jetzt werden sich viele von Ihnen fragen: Aber mein Laptop hat 8 Kerne und da friert doch auch nicht ständig der RAM ein? Stimmt. In Desktop-PCs, also bei SMP-Architekturen, löst die Hardware das Problem mit gigantischem Aufwand selbst. Das nennt sich Cache-Snooping. Die Kerne überwachen sich gegenseitig. Wenn Core 2 auf den Bus zugreift, ruft Core 1: Stopp, ich habe dazu noch neuere Daten in meinem Cache!, schreibt diese schnell in den RAM und lässt Core 2 dann weiterlesen. Aber diese Schnüffelei auf dem Bus kostet Strom, belegt massiv Platz auf dem Silizium und verursacht schwankende Verzögerungen auf dem Bus. Für einen Echtzeit-Controller wie den STM32, bei dem es um Nanosekunden geht, ist Hardware-Snooping absolut inakzeptabel. Wir als Entwickler müssen das Problem selbst lösen.

## Folie

40 / 55

### Software-Lösung 1: Manuelles Management

- Wir zwingen den M7 per C-Code, seinen Cache zu leeren
- Die `core_cm7.h`-Bibliothek bietet dafür spezielle CMSIS-Befehle:
  - `SCB_CleanDCache_by_Addr(...)`: Nimm diese Variable aus dem L1-Cache und schreibe sie zwingend sofort in den SRAM durch! (nach dem Schreiben)
  - `SCB_InvalidateDCache_by_Addr(...)`: Verwirf diese Variable im L1-Cache! Zwing die CPU, sie beim nächsten Mal direkt aus dem SRAM zu lesen!

## Notizen

Die erste Möglichkeit, dieses Problem zu lösen, ist manuelles Cache-Management im C-Code. ARM liefert dafür CMSIS-Kern-Funktionen. Wenn der M7 fertig geschrieben hat, rufen wir `SCB_CleanDCache_by_Addr` auf. Das befiehlt der CPU: Nimm genau diesen Speicherbereich aus dem L1-Cache und schreibe ihn gewaltsam und sofort in den echten SRAM. Umgekehrt, bevor der M7 eine Variable liest, auf die der M4 eventuell geschrieben hat, rufen wir `SCB_InvalidateDCache_by_Addr` auf. Das erklärt die Cache-Line für ungültig und zwingt die M7-Hardware, den Wert beim nächsten Zugriff physikalisch neu aus dem SRAM zu holen.

## Folie

41 / 55

### Nachteile der Software-Lösung

```
// Beispielcode M7:
shared_data.m7_ready = 1;
// Manuelles Durchschreiben!
SCB_CleanDCache_by_Addr( (uint32_t*)&shared_data,
                          sizeof(SharedData_t) );
```

- Warum wir das im Labor NICHT machen:
  - Man vergisst es extrem leicht → schwer zu findende Bugs
  - Der Cache arbeitet immer in 32-Byte-Blöcken (Cache-Lines). Wenn Ihre Datenstruktur nicht exakt auf 32 Byte ausgerichtet ist, invalidieren Sie versehentlich auch benachbarte Variablen!
  - Es kostet unnötig CPU-Taktzyklen

## Notizen

Das manuelle Management funktioniert, ist aber in der Praxis fehleranfällig. Schauen Sie sich den Code an. Sie müssen nach jedem Schreibvorgang diese Clean-Funktion aufrufen. Vergessen Sie es nur an einer einzigen Stelle im Code, haben Sie Ihren Heisenbug. Viel schlimmer ist das 32-Byte-Problem. Ein L1-Cache reinigt nie einzelne Bytes, er operiert immer auf 32-Byte-Blöcken. Wenn Ihr Struct nur 12 Byte groß ist und Sie es invalidieren, löscht die CPU auch die 20 benachbarten Bytes im Cache aus dem RAM. Liegt dort eine völlig unbeteiligte Variable, die die CPU gerade berechnet hat, vernichten Sie Daten! Wir werden diese Software-Lösung heute deshalb nicht verwenden.

## Hardware-Lösung 2 (Best Practice): Die MPU

- Wir entziehen dem Cache einfach die Erlaubnis, diesen spezifischen RAM-Bereich (SRAM4) zu puffern!
- Das Werkzeug dafür ist die MPU (Memory Protection Unit) im Cortex-M7
- Die MPU ist ein Hardware-Modul, das Zugriffsrechte und Cache-Regeln für bestimmte Speicherregionen definiert
- Unsere Konfiguration:
  - Wir definieren eine MPU-Region über den Bereich des SRAM4 (0x38000000)
  - Wir setzen die Eigenschaft auf *Disable Cache / Non-Cacheable* (ARM: oft TEX/C/B so gesetzt, dass kein Caching erfolgt)

### Notizen

Es gibt einen besseren Weg: Wir lösen das Problem wieder hardwareseitig, aber ohne Snooping. Wir nutzen die Memory Protection Unit, die MPU des Cortex-M7. Die MPU ist eine Art Türsteher im Prozessorkern. Wir können der MPU sagen: Pass auf, der Bereich ab 0x38000000 gehört zum Shared Memory. Dieser Bereich ist tabu für den Cache! Wir markieren diesen speziellen Teil des RAMs als Non-Cacheable (in CubeMX oft: Disable Cache). Sobald wir das tun, wird die Hardware den L1-Cache für alle Variablen aus diesem Struct schlichtweg umgehen.

## Was bewirkt die MPU?

- Wenn der M7 nun `shared_data.m7_ready = 1;` ausführt:
  - Die MPU sieht die Adresse 0x38000000
  - Sie weiß: Cache ist hier verboten!
  - Sie leitet den Schreibbefehl direkt über die Bus-Matrix an den SRAM4 durch
- Vorteile:
  - Kein C-Code (Clean/Invalidate) mehr nötig!
  - Cache-Kohärenz ist zu 100 % physikalisch garantiert
  - Der M4 sieht die neuen Daten sofort in Echtzeit
- Nachteil: Zugriffe auf das Struct sind langsamer (Bus-Latenz), aber für ein paar Ping-Pong-Flags ist das irrelevant

### Notizen

Was passiert nun zur Laufzeit? Der M7 schreibt `m7_ready = 1`. Der Schreibbefehl geht an den Kern. Der Türsteher, die MPU, sieht die Adresse und leitet den Befehl am L1-Cache vorbei direkt in die AXI-Bus-Matrix bis zum SRAM4. Die 1 landet sofort und ohne Umwege physikalisch im RAM. Das Schöne

daran: Unser C-Code bleibt absolut sauber. Wir müssen keine Clean-Funktionen mehr aufrufen. Der einzige Nachteil ist, dass dieser direkte Zugriff 5 Takte statt 1 Takt dauert. Aber für die wenigen Bytes an Status-Flags in der Multicore-Kommunikation ist diese minimale Latenz absolut zu verschmerzen.

## Folie

44 / 55

### Laborvorbereitung: Der Ping-Pong Datenaustausch

- Ziel für heute Nachmittag:
  - Beide Kerne wecken
  - Shared Memory konfigurieren
  - Daten hin- und herspielen (Ping-Pong mit Handshake-Flags)

## Notizen

Damit sind wir bei der Vorbereitung für das heutige Labor angekommen. Wir werden den STM32H755 in seiner ganzen asymmetrischen Pracht nutzen: Shared Memory, Linker-Sektion, MPU und ein kontrolliertes Handshake zwischen M7 und M4.

## Folie

45 / 55

### Schritt 1: CubeMX Dual-Core Setup

- Starten Sie ein Projekt für das Board (mit beiden Cores M7 & M4)
- Unter Middleware → FreeRTOS:
  - Aktivieren Sie FreeRTOS ausschließlich für den Cortex-M4
  - Den M7 lassen wir komplett Bare-Metal laufen! (Asymmetric OS-Design)

## Notizen

Legen Sie ein neues Projekt an und aktivieren Sie beide Kerne. Wir setzen heute ein klassisches Industrie-Pattern um: FreeRTOS ausschließlich auf dem M4-Kern für die harte Echtzeit. Den M7-Kern lassen wir als hochperformantes Bare-Metal-Programm völlig ohne Betriebssystem laufen.

## Schritt 2: Den Cortex-M4 wecken

- In der generierten Struktur öffnen Sie die `main.c` im Ordner CM7
- Fügen Sie ganz am Anfang der `main()` den Weckruf ein!

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    // WICHTIG: M4 aus dem Hardware-Reset befreien!
    HAL_PWREx_ReleaseCore(PWR_CORE_CPU2);

    // ... restlicher Code des M7
}
```

### Notizen

Der erste Programmierschritt im Labor ist extrem wichtig: Öffnen Sie den Ordner für den CM7. In dessen `main`-Funktion hat der Codegenerator bereits die HAL-Initialisierung und den System-Clock konfiguriert. Darunter müssen Sie den M4-Kern aufwecken, indem Sie die Hardware-Reset-Linie lösen. Rufen Sie `HAL_PWREx_ReleaseCore` auf. Ab dieser Mikrosekunde laufen beide Kerne gleichzeitig.

## Schritt 3: Shared Memory Struct anlegen

- Legen Sie eine neue Datei `shared_data.h` im `Common/Inc`-Ordner an
- Binden Sie diese Header-Datei in der `main.c` beider Kerne ein

```

#ifndef SHARED_DATA_H
#define SHARED_DATA_H

#include <stdint.h>

typedef struct {
    uint8_t m7_ready;
    uint8_t m4_ready;
    uint32_t payload_value;
} SharedData_t;

// Deklaration mit Sektions-Zuweisung
extern __attribute__((section(".shared_memory")))
    volatile SharedData_t sd;

#endif

```

## Notizen

In Schritt 3 definieren wir unsere Sprache. Da beide Kerne völlig getrennte C-Projekte sind, müssen wir eine Header-Datei im Common-Ordner anlegen, auf den beide Zugriff haben. Wir definieren unser Struct `SharedData_t` mit den Bereit-Flags und einem Payload. Darunter deklarieren wir die Variable `sd` als `extern`, binden sie fest an die `.shared_memory`-Sektion und markieren sie als `volatile`. Damit haben beide Kerne dasselbe logische Verständnis der Datenstruktur. Hinweis: Die Definition (ohne `extern`) gehört in genau eine `.c`-Datei, typischerweise im Common-Ordner.

## Folie

48/55

### Schritt 4: Das Linker-Script anpassen

- Öffnen Sie in beiden Projekten (CM7 und CM4) die Datei `STM32H755..._FLASH.ld`
- Fügen Sie im MEMORY-Block den SRAM4 hinzu (CubeMX nennt ihn oft `RAM_D3`; früher auf den Folien: `SHARED_RAM` — entscheidend ist die Adresse):

```
RAM_D3 (xrw) : ORIGIN = 0x38000000, LENGTH = 64K
```

- Fügen Sie im SECTIONS-Block unsere Sektion ein (Achtung auf `NOLOAD`!):

```

.shared_memory (NOLOAD) :
{
    . = ALIGN(4);
    *(.shared_memory)
} > RAM_D3

```

## Notizen

Nun müssen wir den Worten im C-Code auch Taten im Linker-Script folgen lassen. Suchen Sie die `.ld`-Dateien im Projektbaum. Fügen Sie im `MEMORY`-Block den D3-RAM ein, beginnend bei `0x38000000`. Dann scrollen Sie runter in den `SECTIONS`-Block und fügen unsere Sektion `.shared_memory` ein. Vergessen Sie das `NOLOAD` nicht, sonst überschreibt der M4-Bootcode den SRAM4 mit Nullen und zerschießt die Daten, die der M7 vielleicht schon geschrieben hat. Sie müssen diese Änderungen zwingend in den Linker-Scripten beider Kerne vornehmen!

## Folie

49/55

### Schritt 5: Die Ping-Pong Logik (Cortex-M4)

- In der FreeRTOS-Task des M4:

```
void StartDefaultTask(void *argument) {
    for(;;) {
        // 1. Warten bis M7 Daten bereitstellt
        while(sd.m7_ready == 0) {
            osDelay(1); // Task schlafen, CPU nicht blockieren
        }

        // 2. Daten verarbeiten
        sd.m7_ready = 0; // Quittieren
        sd.payload_value += 1;

        // 3. Antwort an M7 signalisieren
        sd.m4_ready = 1;
    }
}
```

## Notizen

Kommen wir zur eigentlichen Logik. Auf dem M4 läuft unser FreeRTOS. In der generierten Task bauen wir eine Warteschleife. Solange der M7 noch keine Daten geliefert hat (`m7_ready == 0`), schicken wir die Task mit `osDelay(1)` schlafen. So belasten wir die Echtzeit-Umgebung des M4 nicht mit ewigem Busy-Polling. Sobald das Flag auf 1 springt, wachen wir auf. Wir löschen das M7-Flag (Quittierung), addieren eine 1 auf den Payload-Wert und setzen unser eigenes Bereit-Flag `m4_ready` auf 1. Der Ball liegt wieder beim M7.

## Schritt 6: Die Ping-Pong Logik (Cortex-M7)

- In der Super-Loop (`while (1)`) des M7:

```
// Initiale Übergabe
sd.payload_value = 0;
sd.m7_ready = 1;

while (1) {
    // 1. M7 hat kein OS! Polling ist hier (als Demo) ok.
    if(sd.m4_ready == 1) {

        printf("M4 hat geantwortet: %lu\n",
               (unsigned long)sd.payload_value);

        sd.m4_ready = 0; // Quittieren
        sd.payload_value += 1;

        HAL_Delay(500); // Demo: Lesbarkeit am Terminal
        sd.m7_ready = 1; // Ball zurück zu M4!
    }
}
```

### Notizen

Das Gegenstück auf dem M7 ist reines Bare-Metal. Wir initiieren das Spiel vor der While-Schleife, indem wir den Payload auf 0 setzen und den Ball losspielen (`m7_ready = 1`). In der Super-Loop pollen wir das Flag des M4. Da wir auf dem M7 kein Betriebssystem haben, ist das Polling hier in Ordnung. Wenn der M4 geantwortet hat, drucken wir den Wert über UART aus, quittieren das Flag, addieren wieder eine 1, machen absichtlich ein `HAL_Delay` (nur Demo, Bare-Metal!), damit wir die Ausgabe am Terminal mitlesen können, und spielen den Ball zurück zum M4.

## Schritt 7: MPU im M7 konfigurieren

- Ohne diese Konfiguration wird der Code aus Schritt 6 scheitern!
- Gehen Sie im CubeMX Configurator auf: System Core → CORTEX\_M7 → Reiter MPU Control
- Aktivieren Sie Region 0:
  - Base Address: 0x38000000
  - Size: 64 KB
  - Access Permission: ALL ACCESS
  - TEX / Shareable / Cacheable: Disable Cache (Non-Cacheable)
- Code neu generieren!

### Notizen

Würden wir den Code jetzt flashen, würde absolut nichts passieren. Das System friert ein — wegen des Cache-Kohärenz-Problems. Um das zu beheben, müssen wir in den Configurator der CubeIDE. Unter System Core finden Sie die Cortex-M7-Einstellungen. Dort gehen Sie in das Memory Protection Unit Control. Wir legen eine neue Regel an: Für den Bereich beginnend bei 0x38000000 mit einer Größe von 64 KB stellen wir das Caching-Verhalten auf Disable Cache. Generieren Sie den Code. Ab jetzt umgeht die M7-Hardware den L1-Cache komplett, wenn sie auf unser `sd-Struct` zugreift.

## Flashen & Debuggen im Labor

- Klicken Sie im CM7-Projekt auf den kleinen Pfeil neben dem Käfer-Icon (Debug) und wählen Sie „Debug Configurations“
- Die CubeIDE hat automatisch eine Startup-Gruppe angelegt
- Achten Sie darauf, dass beide `.elf`-Dateien (CM7 und CM4) ausgewählt sind
- Ein Klick lädt erst den M7 in den Flash (Bank 1), danach den M4 (Bank 2) und stoppt beide an ihrer jeweiligen `main()`
- Drücken Sie auf beiden Konsolen auf Play (Resume)

### Notizen

Das Flashen eines Dual-Core-Projekts bedarf eines kleinen Tricks in Eclipse. Sie dürfen nicht einfach blind auf Run klicken, sonst flashen Sie vielleicht nur den M7. Gehen Sie in die Debug Configurations. ST hat dort meist schon eine Multicore-Startgruppe angelegt. Vergewissern Sie sich, dass die Checkboxes für beide Firmware-Dateien, also `CM7.elf` und `CM4.elf`, gesetzt sind. Wenn Sie starten, brennt der ST-Link zuerst Programm 1 und dann Programm 2 in den Chip. Dann halten beide Kerne in der ersten Zeile Code an. Sie müssen dann oben in der Taskleiste beide Kerne separat auf Resume drücken, damit

das Ping-Pong-Spiel beginnt.

Folie

53 / 55

### Mutwilliger Fehler: MPU deaktivieren

- Wenn Ihr System läuft und der UART fröhlich zählt:
- Gehen Sie in den Code und kommentieren Sie die MPU-Initialisierung aus:

```
/* HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT); */
```

- Die Beobachtung:
  - Der M7 gibt initial 0 aus und friert dann für immer ein
  - Der M4 wartet ewig im `osDelay(1)`
- Sie haben den berühmigten Cache Coherency Failure live am Debugger!

### Notizen

Wenn Ihr Terminal im Labor anfängt, im 500 ms-Takt die Zahlen hochzuzählen, haben Sie es geschafft. Beide Kerne reden miteinander. Um sich aber zu beweisen, dass die Theorie stimmt, machen wir einen mutwilligen Fehler. Gehen Sie in die `main.c` des M7 und kommentieren Sie die Zeile aus, in der die MPU aktiviert wird (`HAL_MPU_Enable`). Flashen Sie das Programm neu. Das Terminal wird stumm bleiben. Der M7 schreibt in seinen L1-Cache, der M4 liest aus dem SRAM4 die alten Nullen aus. Beide hängen in ihren While-Schleifen fest. Sie haben gerade den berühmigtsten Architektur-Fehler der Multicore-Entwicklung live erzeugt.

Folie

54 / 55

### Die Gefahr für nächste Woche (HSEM)

- Unser Ping-Pong heute nutzt ein strenges Handshake-Protokoll (`m7_ready`, `m4_ready`). Jeder Kern weiß genau, wann er schreiben darf
- Was passiert aber...
  - wenn beide Kerne asynchron arbeiten und gleichzeitig `sd.payload_value` überschreiben wollen?
- Ein FreeRTOS-Mutex hilft hier nicht! (Der M7 hat ja gar kein FreeRTOS!)
- Wir brauchen hardware-basierte Schlösser, die über Core-Grenzen hinweg funktionieren (HSEM)
- → Thema für Vorlesung 8!

### Notizen

Das heutige Labor ist ein kontrolliertes Ping-Pong. Die Fahne `m7_ready` sorgt dafür, dass immer nur ein Kern zur selben Zeit in das Struct schreibt. Aber was passiert nächste Woche, wenn beide Kerne

völlig asynchron und unabhängig voneinander auf denselben Payload-Wert zugreifen wollen? Dann haben wir eine Multicore Race Condition. Unser schöner FreeRTOS-Mutex aus Woche 4 hilft uns hier keinen Millimeter weiter. Der Mutex existiert ja nur im RAM des M4-Kerns, der Bare-Metal-M7 weiß davon gar nichts. Wir benötigen ein Schloss, das auf Hardware-Ebene in Silizium gegossen ist und über Bus-Grenzen hinweg funktioniert. Das nennt sich Hardware Semaphores (HSEM) und wird das große Thema in Vorlesung 8 sein.

Folie

55 / 55

## Zusammenfassung des Tages

- AMP ist der Standard für harte Echtzeit-Multicore-Systeme, um Interferenzen zwischen Betriebssystemen zu vermeiden
- Der Cortex-M4 übernimmt oft das RTOS, der Cortex-M7 (Bare-Metal) kümmert sich um den Durchsatz
- Die Adress-Zuordnung von geteilten Daten muss hart über `.ld`-Linker-Skripte erzwungen werden (`.shared_memory`)
- Cache-Kohärenz zerschießt die Kommunikation, wenn schnelle Kerne (M7) Write-Back-Caches nutzen
- Die Lösung ist die Deaktivierung des Caches für den geteilten RAM per Hardware (MPU)

## Notizen

Fassen wir zusammen. Sie kennen nun den fundamentalen Architektur-Unterschied zwischen dem Laptop zu Hause (SMP) und dem Steuergerät im Labor (AMP). Sie können den Boot-Ablauf eines Dual-Core-Chips orchestrieren. Sie verstehen, wie man per C-Attribut Daten in spezifische SRAM-Sektoren zwingt. Und das Wichtigste: Sie haben verstanden, warum ein L1-Cache uns die Datenkonsistenz zerstört und wie wir das durch die Memory Protection Unit auf Hardware-Ebene sauber und professionell beheben können. Damit sind Sie perfekt gewappnet für das Labor am Nachmittag!